Stream Joins on Shared-Memory Multicore Architectures

ABSTRACT

Different from conventional join algorithm, which processes a large batch of data at once, *stream join* emphasizes at producing results early and continuously, which is mandatory for tasks such as interactive data visualization. Pioneer works mostly focus on single-thread execution efficiency taking care of out-of-memory issue, while recent works gearing towards shared-nothing architecture using a cluster of commodity machines. Little attention has been paid on the efficiency of stream joins on modern shared-memory multicore architectures, leaving an unpleasant literature gap. In particular, more cores are being put on the same die, on-chip cache hierarchies are getting larger, deeper, and complex, and memory capacity increases immensely with non-uniform memory access (NUMA) architecture. Those changes require us to revisit the design of main-memory stream join algorithms.

In this paper we study the efficiency and trade-off of different stream join algorithms on modern scale-up architectures. Specifically, we implement different designs of stream join algorithms on a common test-bed to conduct detailed profiling studies. The analysis and comparisons in the paper show that: a) existing hardware-ignorant implementation of stream joins poorly utilizes (only ~xx%) modern multicore processors; b) the lack of NUMA-awareness causing further performance degradation up to xx%. We present our initial efforts on resolving those performance issues, and present fastest implementation of stream join to date. In processing xxx tuples on a modern 40-core machine, it can generate 25th, 50th, 75th and 100th results in xx, xx, xx, and xx ms, respectively.

ACM Reference Format:

. 2019. Stream Joins on Shared-Memory Multicore Architectures. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Operations on large datasets may take a long time to complete, especially for heavy operations such as join. Taking trajectory similarity join as an example. The objective is to join all "similar" pairs of trajectories from two sets of trajectories. State-of-the-art implementations [18] of trajectory similarity join are still mostly blocking-based, i.e., return answers only after a potentially long processing period. Such lack of interactivity and flexibility leads to unpleasant waiting time for user and prevents interactive tasks, e.g., interactive trajectory mining [2].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2019 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

The need to obtain joining results "early" (before having read an entire input) has been long identified in the community before. An important line of related study is on sliding window-based join [12], which cuts input stream into finite windows, and joins over a recent window of tuples. While such window-based relational model significantly simplifies stream processing, it is ineffective for emerging application demands that require maintaining large historical states [4, 6]. Therefore, in this work, we focus on *full-history* stream-join processing, where input size is bounded and is involved entirely during join-processing.

Since the pioneer work of symmetric hash join [17], a number of stream join algorithms (also called online/progressive/nonblocking joins) have been proposed and they are mostly focus on single-thread execution efficiency and can be further classified into two categorizes: 1) those taking care of out-of-memory issue [5, 8, 11, 14, 16] and 2) ripple join and its variants [7, 9] emphasize on providing higher statistical quality of intermediate aggregation results. To cope with the rapid growth of data sets, much effort has been recently put into designing distributed and parallel join algorithms [6]. However, existing works on parallelizing stream join gearing towards shared-nothing architecture using a cluster of commodity machines and overlook the hardware utilization of each machine. Furthermore, we observe that they are mostly built around two key design aspects:

- **D1: Data Partition-based Parallelism.** Parallel stream join is often realized via data partition. Specifically, input stream is partitioned among cores and each core is assigned a "local joiner" to process the subset of input data in parallel. Different partition models [6, 10, 13, 15] haven been proposed but little work has compared them in a common test-bed on shared-memory multicore architecture.
- **D2: Hardware-ignorant Local Joiner.** Each "local joiner" is often implemented by simply adopting an existing single-threaded algorithm (e.g., symmetric hash join [17] and ripple join [7]). The basic idea is that each new tuple from one relation is joined with the tuples of the other, stored (e.g., in the hash table), and the output for the joined tuple is then generated. Despite its simplicity, such hardware-ignorant algorithm may result in sub-optimal performance on modern multicore processors with complex hardware architectures.

Multicore architectures are often characterized as "emerging", but that is no longer the case. Thanks to the great efforts from computer architecture community, multicore processors are increasingly available but fully utilizing their computation power is still notoriously challenging. More CPU cores are being put on the same die. Subsequently, the on-chip cache hierarchies that support these cores are getting larger, deeper, and more complex. Furthermore, as modern machines scale to multiple sockets, non-uniform memory access (NUMA) becomes an important performance factor for data management systems (e.g., [19]). For example, recent NUMA systems have already supported hundreds of CPU cores and multi-terabytes of memory [1]. However, there

is a lack of detailed studies on profiling the above common design aspects of stream joins on modern architectures.

In this paper, we aim to offer a better understanding of how different designs of stream joins interact with modern processors under varying workloads. In particular, we perform a comprehensive study on varies types of 1) data partition models and 2) main-memory local joiner implementations. For each aspect, we describe the state-of-the-art implementations and their corresponding modification to adapt to shared-memory architectures and discuss their trade-offs.

Through detailed profiling studies on a four-socket machine, we make the following key observations.

First, existing hardware-ignorant implementation of stream joins (**D2**) poorly utilizes modern multicore processors. Based on our profiling results, a significant portion (~xx%) of the total execution time is wasted due to xxx, which is caused by xxx. xxx *Second,* the current design of data partition (**D1**) either 1) does not satisfy the low-latency processing requirement of stream join or 2) lack of NUMA-awareness causing a performance degradation up to xx% because of local joiner straggler. xxx

Addressing the above-mentioned issues should allow stream joins to exploit modern scale-up architectures. First, we present a NUMA-aware workload distribution scheme (**O1**) to make data partition aware of remote memory access overhead. Second, we implement hardware-conscious local joiners (**O2**) to optimize stream join processing at each core.

O1: **NUMA-aware Data Partition.** xxx
O2: **Hardware-conscious Local Joiners.** xxx

To the best of our knowledge, this is the first detailed study of different design aspects of stream joins on modern multicore architectures. Improving its efficiency on shared-memory multicore architectures is also beneficial for scale-out: it either offers a better performance with the same number of machines or achieve the same performance requirement with lesser machines.

The remainder of this paper proceeds as follows: xxx

2 PRELIMINARIES

In this section, we introduce background of stream join and modern scale-up servers.

2.1 Stream Joins

In this paper, we focus on **two-way theta stream join**, that is the algorithm joins two datasets R and S over a binary predict P and continuously produce a list of pair of tuples. As pointed out by prior works [6], the key demands including (i) joins over very large volumes of data; (ii) producing partial results as earlier as possible without sacrificing overall efficiency; (iii) maintain large state, which potentially depend on the complete history of previously processed tuples. This is a very useful operation in many use cases such as online data mining and interactive query processing. For example, for quickly presenting initial/preliminary results to users or downstream application pipelines.

2.2 Modern Scale-up Servers

Modern machines scale to multiple sockets with non-uniform-memory-access (NUMA) architecture. Each socket has its own

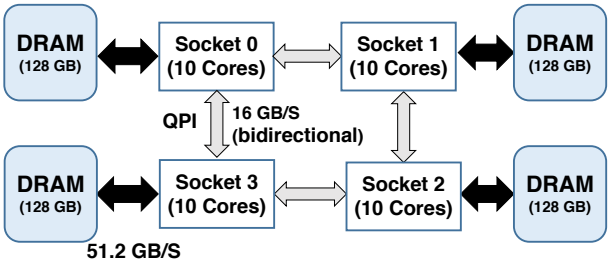

Figure 1: NUMA topology and peak bandwidth of our sever.

"local" memory and is connected to other sockets and, hence to their memory, via one or more links. Therefore, access latency and bandwidth vary depending on whether a core is accessing "local" or "remote" memory. Such NUMA effect requires ones to carefully align the communication patterns accordingly to get good performance.

As modern machines scale to multiple sockets, non-uniform memory access (NUMA) brings more performance issues. Figure 1 illustrates the NUMA topology of our sever with four sockets. Each CPU socket has its local memory, which is uniformly shared by the cores on the socket. Sockets are connected by a much slower (compared to local memory access) channel called Quick Path Interface (QPI).

3 JOIN PARTITIONING SCHEME

The costly nature of stream join and the stringent response time requirements of stream application has created significant interest in accelerating stream join. In this section, we discuss existing or adopted methods to parallelize stream joins.

The most widely adopted approach for parallelizing stream joins is to partition input data and then apply existing single-threaded stream join algorithm on each subset of datasets. In particular, the join-matrix model [6] and the join-biclique model [10] are two representative approaches to deal with the scalable stream join processing. Intuitively, the join matrix model design a join between two datasets R and S as a matrix, where each side corresponds to one relation. Alternatively, the join-biclique model is to organize the processing units as a complete bipartite graph, where each side corresponds to a relation. It is superior in memory efficiency, but is sensitive to the consumption of network bandwidth for tuple routing.

Adopting both models to parallelize stream joins on multicore is straightforward as we can simply treat compute node as CPU cores. This approach is proposed primarily for distributed execution environment, and does not depend on any hardware-specific parameters.

4 IN-MEMORY STREAM JOINER

Hardware-conscious Parallelism. As discussed, both hashing and sorting-based stream join algorithms have been proposed in the literature. However, prior algorithms are designed for single-thread execution. Witnessing the advancement from hardware architectures, many recent research efforts are devoted into

parallelizing stream joins with hardware-awareness. However, they have not been considered in the literature for stream joins.

5 EXPERIMENT

In this section, we first present the evaluation goals of this study. Next, we introduce our targeting machine specification, followed by testing workloads.

Evaluation Goals.

System. Experiments are run on an Intel Sandy Bridge with a 256- bit AVX instruction set. It has a four-socket configuration, with each CPU socket containing 8 physical cores and 16 thread contexts by the help of the hyper-threading. Cache sizes are 32 KiB for L1, 256 KiB for L2, and 20 MiB L3 (the latter shared by the 16 threads within the socket). The cache line size of the system is 64 bytes. TLB1 contains 64/32 entries when using 4 KiB/2 MiB pages (respectively) and 512 TLB2 entries (page size 4 KiB). Total memory available is 512 GiB (DDR3 at 1600 MHz). The system runs Debian Linux 7.0, kernel version 3.4.4-U5 compiled with transparent huge page support and it uses 2 MiB VM pages for memory allocations transparently. This has been shown to improve [3, 4]. Therefore, we assume the availability of large page support in the system. The code is compiled with gcc 4.7.2 using -O3. Experiments using Intel's icc compiler did not show any notable differences, qualitatively or quantitatively.

5.1 Benchmarks

To facilitate comparisons with existing results, we use similar workloads to those of Kim et al. [15], Albutiu et al. [2] and Balkesen et al. [4]. They all assume a column-oriented storage model and joins are assumed to be evaluated over narrow (8- or 16-byte) hkey, payloadi tuple configurations. To understand the value of data parallelism using vectorized instructions, we assume key and payload are four bytes each. The workloads used are listed in Table 2. All attributes are integers, but AVX currently only supports floating point; therefore we treat integer keys as floats when operating with AVX.1 There is a foreign key relationship from S to R. That is, every tuple in S is guaranteed to find exactly one join partner in R. Most experiments (unless noted otherwise) assume a uniform distribution of key values from R in S.

Two datasets are first loaded in one socket.

5.2 Analysis of Data Partition

5.3 Analysis of Local Joiners

5.4 Put It All Together

6 DISCUSSION

7 RELATED WORK

In this section, we discuss prior works related to this study.

Online Joins.

Window Stream Joins. Stream processing is emerged as a popular solution to handle "big data streams". To handling infinite input streams, a widely accepted compromise is cut stream into finite slices (called windows) and a family of window-based stream operations were proposed in the literature []. Kang et al. [] described the first window stream join implementation with a tree-step procedure. An earlier work from Gedik et al. [], called CellJoin, attempt to parallelize stream join on Cell processor. Subsequently, Handshake join [], SplitJoin [12] are proposed to better utilize modern multicore architectures. However, their primary concern is to handle sliding window progress and the parallelism is often gained from processing multiple windows simultaneously [12]. In contrast, we concern bounded stream joins, which can be also viewed as infinite window stream join, which makes window stream join algorithms ineffective in our context.

Parallel/Distributed Joins. Motivated by the increasingly widely available parallel computing resources, much recent effort has been put into designing distributed and parallel join algorithms to cope with the rapid growth of data sets. The goal is often to find a scheme that minimizes overall join processing time through wisely managing the balance between computation and communication cost [3]. Similarity joins in the context of data mining are an excellent example. They are essential for basic operations like clustering and outlier detection. Majority of the techniques for similarity joins are still blocking-based, and therefore return answers only after a potentially long processing period despite the great efforts from the community to parallelizing join processing.

8 CONCLUSION

REFERENCES

[1] Sgi® uv 300™ – the most powerful in-memory supercomputer, http://www.comnetco.com/sgi-uv300-the-most-powerful-in-memory-supercomputer/, 2019.

[2] J. Ang, T. Fu, J. Paul, S. Zhang, B. He, T. S. D. Wenceslao, and S. Tan. Trav: an interactive trajectory exploration system for massive data sets. In *Proceedings of the Fifth IEEE International Conference on Multimedia Big Data*, 2019.

[3] C. Balkesen, G. Alonso, J. Teubner, and M. T. Özsu. Multi-core, main-memory joins: Sort vs. hash revisited. *Proc. VLDB Endow.*, 7(1):85–96, Sept. 2013.

[4] R. Castro Fernandez, M. Migliavacca, E. Kalyvianaki, and P. Pietzuch. Integrating scale out and fault tolerance in stream processing using operator state management. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, SIGMOD '13, pages 725–736, New York, NY, USA, 2013. ACM.

[5] J.-P. Dittrich, B. Seeger, D. S. Taylor, and P. Widmayer. Progressive merge join: A generic and non-blocking sort-based join algorithm. In *Proceedings of the 28th International Conference on Very Large Data Bases*, VLDB '02, pages 299–310. VLDB Endowment, 2002.

[6] M. Elseidy, A. Elguindy, A. Vitorovic, and C. Koch. Scalable and adaptive online joins. *Proc. VLDB Endow.*, 7(6):441–452, Feb. 2014.

[7] P. J. Haas and J. M. Hellerstein. Ripple joins for online aggregation. *SIGMOD Rec.*, 28(2):287–298, June 1999.

[8] R. Lawrence. Early hash join: A configurable algorithm for the efficient and early production of join results. In *Proceedings of the 31st International Conference on Very Large Data Bases*, VLDB '05, pages 841–852. VLDB Endowment, 2005.

[9] F. Li, B. Wu, K. Yi, and Z. Zhao. Wander join: Online aggregation via random walks. In *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, pages 615–629, New York, NY, USA, 2016. ACM.

[10] Q. Lin, B. C. Ooi, Z. Wang, and C. Yu. Scalable distributed stream join processing. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, SIGMOD '15, pages 811–825, New York, NY, USA, 2015. ACM.

[11] M. F. Mokbel, M. Lu, and W. G. Aref. Hash-merge join: A non-blocking join algorithm for producing fast and early join results. In *Proceedings of the 20th International Conference on Data Engineering*, ICDE '04, pages 251–, Washington, DC, USA, 2004. IEEE Computer Society.

[12] M. Najafi, M. Sadoghi, and H.-A. Jacobsen. Splitjoin: A scalable, low-latency stream join architecture with adjustable ordering precision. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, pages 493–505, Denver, CO, June 2016. USENIX Association.

[13] P. Roy, J. Teubner, and R. Gemulla. Low-latency handshake join. *Proc. VLDB Endow.*, 7(9):709–720, May 2014.

[14] Y. Tao, M. L. Yiu, D. Papadias, M. Hadjieleftheriou, and N. Mamoulis. Rpj: Producing fast join results on streams through rate-based optimization. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, SIGMOD '05, pages 371–382, New York, NY, USA, 2005. ACM.

[15] J. Teubner and R. Mueller. How soccer players would do stream joins. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*, SIGMOD '11, pages 625–636, New York, NY, USA, 2011. ACM.

[16] T. Urhan and M. J. Franklin. Xjoin: A reactively-scheduled pipelined join operatorỳ. *Bulletin of the Technical Committee on*, page 27, 2000.

[17] A. N. Wilschut and P. M. G. Apers. Dataflow query execution in a parallel main-memory environment. In *[1991] Proceedings of the First International Conference on Parallel and Distributed Information Systems*, pages 68–77, Dec 1991.

[18] H. Yuan and G. Li. Distributed in-memory trajectory similarity search and join on road network. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1262–1273, April 2019.

[19] S. Zhang, J. He, A. C. Zhou, and B. He. Briskstream: Scaling data stream processing on shared-memory multicore architectures. In *Proceedings of the 2019 International Conference on Management of Data*, SIGMOD '19, pages 705–722, New York, NY, USA, 2019. ACM.